UNIT-IV

TOPIC:- MQTT, MQTT methods and components

OBJECTIVES:- MQTT is a machine to machine internet of things connectivity protocol. It is an extremely lightweight and publish-subscribe messaging transport protocol

OUTCOME:-

MQTT protocol

MQTT stands for **Message Queuing Telemetry Transport**. MQTT is a machine to machine internet of things connectivity protocol. It is an extremely lightweight and publish-subscribe messaging transport protocol. This protocol is useful for the connection with the remote location where the bandwidth is a premium. These characteristics make it useful in various situations, including constant environment such as for communication machine to machine and internet of things contexts. It is a publish and subscribe system where we can publish and receive the messages as a client. It makes it easy for communication between multiple devices. It is a simple messaging protocol designed for the constrained devices and with low bandwidth, so it's a perfect solution for the internet of things applications.

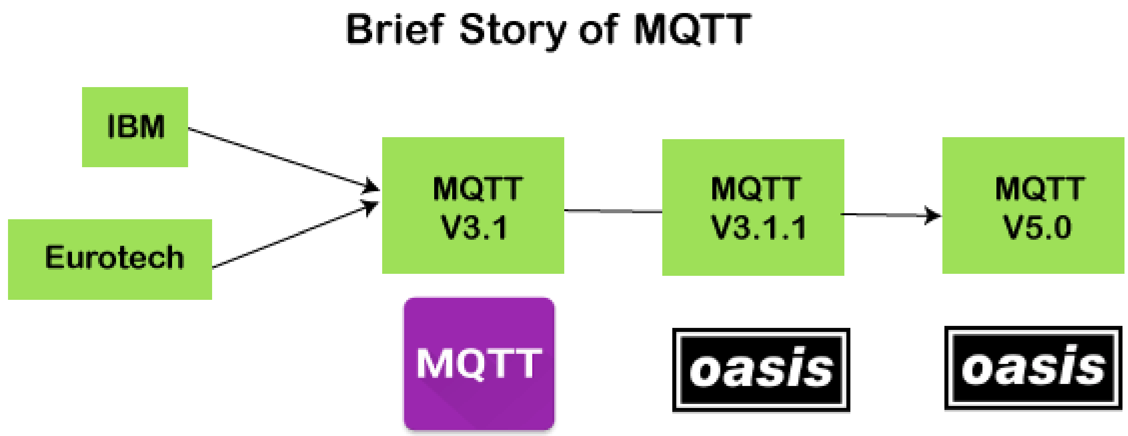
Characteristics of MQTT

The MQTT has some unique features which are hardly found in other protocols. Some of the features of an MQTT are given below:

- It is a machine to machine protocol, i.e., it provides communication between the devices.
- It is designed as a simple and lightweight messaging protocol that uses a publish/subscribe system to exchange the information between the client and the server.
- It does not require that both the client and the server establish a connection at the same time.
- It provides faster data transmission, like how WhatsApp/messenger provides a faster delivery. It's a real-time messaging protocol.

- It allows the clients to subscribe to the narrow selection of topics so that they can receive the information they are looking for.

History of MQTT



The MQTT was developed by Dr. Andy Stanford-Clark, IBM, and Arlen Nipper. The previous versions of protocol 3.1 and 3.1.1 were made available under MQTT ORG. In 2014, the MQTT was officially published by OASIS. The OASIS becomes a new home for the development of the MQTT. Then, the OASIS started the further development of the MQTT. Version 3.1.1 is backward comfortable with a 3.1 and brought only minor changes such as changes to the connect message and clarification of the 3.1 version. The recent version of MQTT is 5.0, which is a successor of the 3.1.1 version. Version 5.0 is not backward, comfortable like version 3.1.1. According to the specifications, version 5.0 has a significant number of features that make the code in place.

The major functional objectives in version 5.0 are:

- Enhancement in the scalability and the large-scale system in order to set up with the thousands or the millions of devices.
- Improvement in the error reporting

MQTT Architecture

To understand the MQTT architecture, we first look at the components of the MQTT.

- **Message**
- **Client**
- **Server or Broker**
- **TOPIC**

Message

The message is the data that is carried out by the protocol across the network for the application. When the message is transmitted over the network, then the message contains the following parameters:

1. Payload data
2. Quality of Service (QoS)
3. Collection of Properties
4. Topic Name

Client

In MQTT, the subscriber and publisher are the two roles of a client. The clients subscribe to the topics to publish and receive messages. In simple words, we can say that if any program or device uses an MQTT, then that device is referred to as a client. A device is a client if it opens the network connection to the server, publishes messages that other clients want to see, subscribes to the messages that it is interested in receiving, unsubscribes to the messages that it is not interested in receiving, and closes the network connection to the server.

In MQTT, the client performs two operations:

In MQTT, the client performs two operations:

Publish: When the client sends the data to the server, then we call this operation as a publish.

Subscribe: When the client receives the data from the server, then we call this operation a subscription.

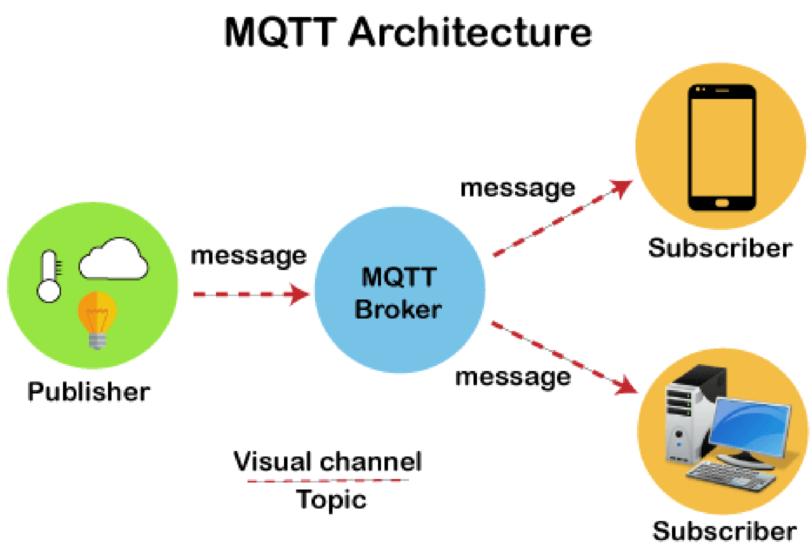
Server

The device or a program that allows the client to publish the messages and subscribe to the messages. A server accepts the network connection from the client, accepts the messages from the client, processes the subscribe and unsubscribe requests, forwards the application messages to the client, and closes the network connection from the client.

TOPIC

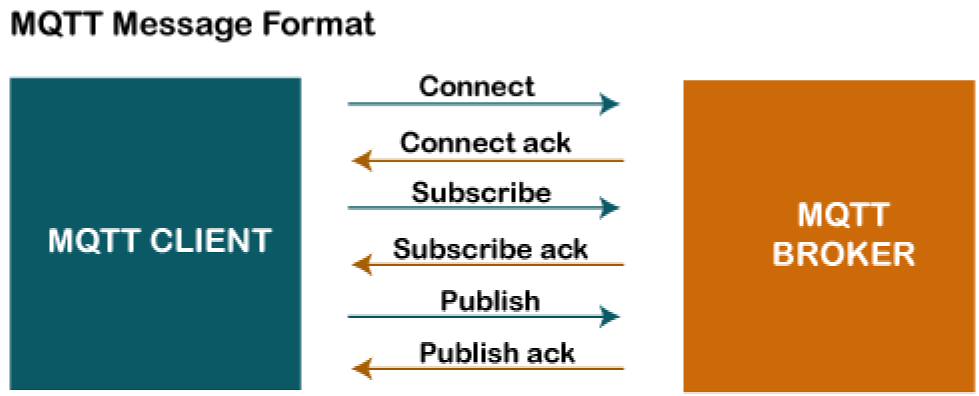
The label provided to the message is checked against the subscription known by the server is known as TOPIC.

Architecture of MQTT



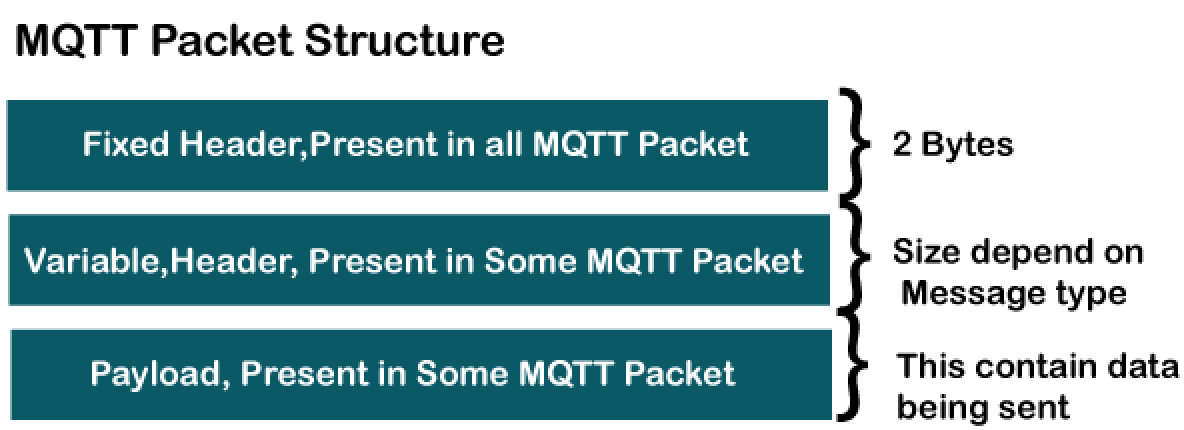
Now we will look at the architecture of MQTT. To understand it more clearly, we will look at the example. Suppose a device has a temperature sensor and wants to send the rating to the server or the broker. If the phone or desktop application wishes to receive this temperature value on the other side, then there will be two things that happened. The publisher first defines the topic; for example, the temperature then publishes the message, i.e., the temperature's value. After publishing the message, the phone or the desktop application on the other side will subscribe to the topic, i.e., temperature and then receive the published message, i.e., the value of the temperature. The server or the broker's role is to deliver the published message to the phone or the desktop application.

MQTT Message Format



The MQTT uses the command and the command acknowledgment format, which means that each command has an associated acknowledgment. As shown in the above figure that the connect command has connect acknowledgment, subscribe command has subscribe acknowledgment, and publish command has publish acknowledgment. This mechanism is similar to the handshaking mechanism as in TCP protocol.

Now we will look at the packet structure or message format of the MQTT.



The MQTT message format consists of 2 bytes fixed header, which is present in all the MQTT packets. The second field is a variable header, which is not always present. The third field is a payload, which is also not always present. The payload field basically contains the data which is being sent. We might think that the payload is a compulsory field, but it does not happen. Some commands do not use the payload field, for example, disconnect message.

REFERENCES:-

SUMMARY:-

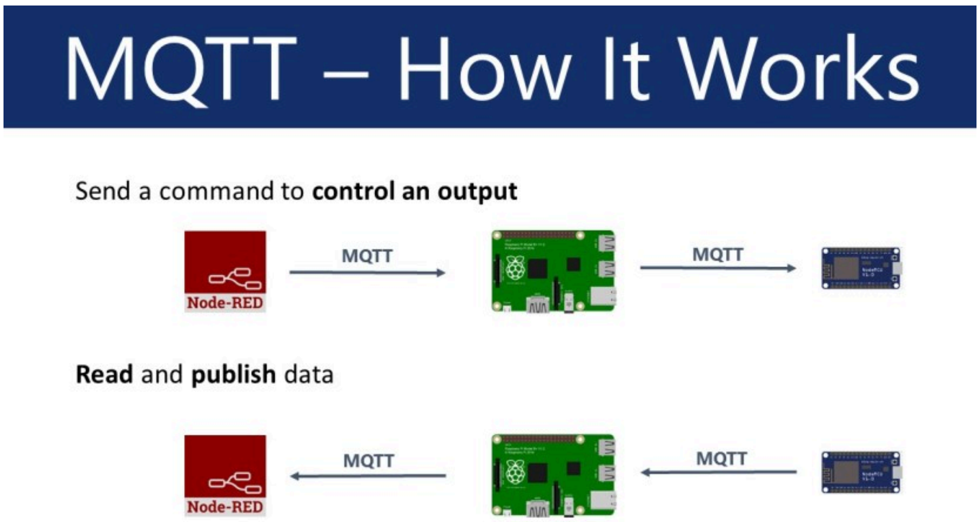
TOPIC:- **MQTT communication, topics and applications, SMQTT**

OBJECTIVES:- MQTT is a simple messaging protocol, designed for constrained devices with low-bandwidth. So, it's the perfect solution for Internet of Things applications

OUTCOME:-

MQTT communication, topics and applications, SMQTT

MQTT stands for **M**essage **Q**ueuing **T**elemetry **T**ransport.



It is a lightweight publish and subscribe system where you can publish and receive messages as a client.

MQTT is a simple messaging protocol, designed for constrained devices with low-bandwidth. So, it's the perfect solution for Internet of Things applications. MQTT allows you to send commands to control outputs, read and publish data from sensor nodes and much more.

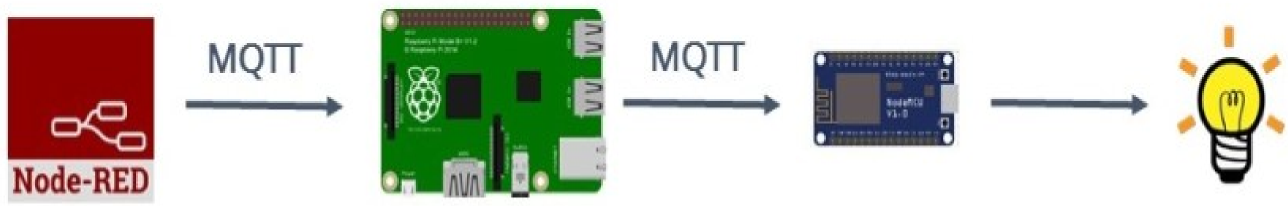
Therefore, it makes it really easy to establish a communication between multiple devices.

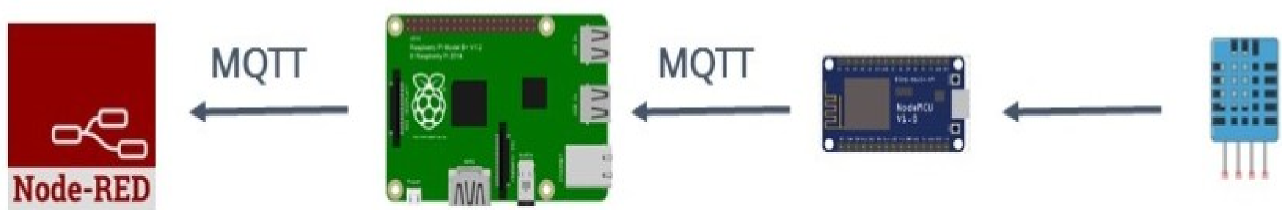
If you like home automation and you want to build a complete home automation system, I recommend downloading my home automation course .

Watch the video tutorial below

High Level Overview

Here’s a quick high level overview of what MQTT allows you to do.





Basic Concepts

In MQTT there are a few basic concepts that you need to understand:

- Publish/Subscribe
- Messages
- Topics
- Broker

Publish/Subscribe

The first concept is the *publish and subscribe* system. In a publish and subscribe system, a device can publish a message on a topic, or it can be subscribed to a particular topic to receive messages

- For example **Device 1** publishes on a topic.
- **Device 2** is subscribed to the same topic as **device 1** is publishing in.
- So, **device 2** receives the message.

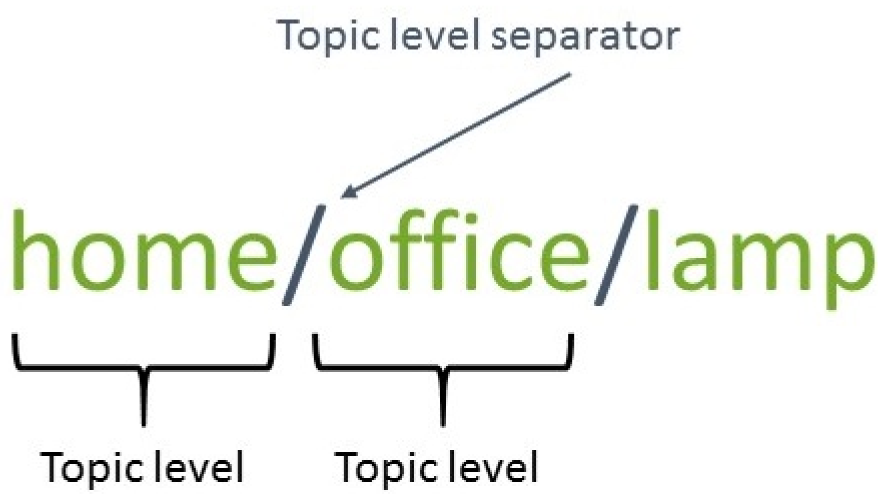
MQTT – Messages

Messages are the information that you want to exchange between your devices. Whether it's a command or data.

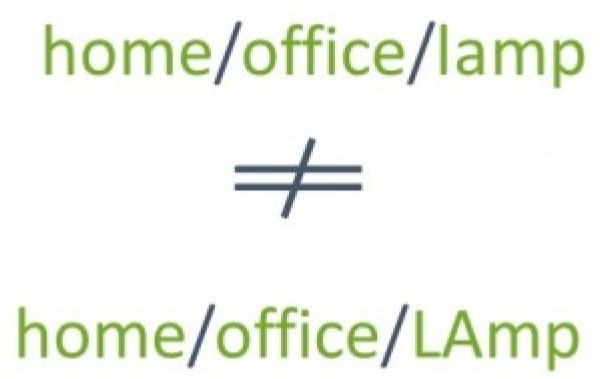
MQTT – Topics

Another important concept are the *topics*. Topics are the way you register interest for incoming messages or how you specify where you want to publish the message.

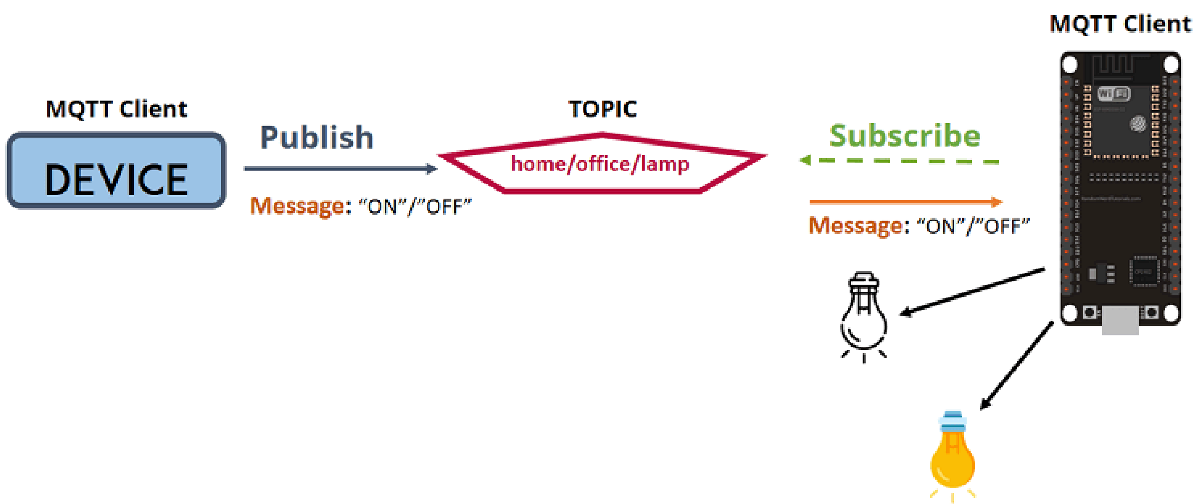
Topics are represented with strings separated by a forward slash. Each forward slash indicates a topic level. Here's an example on how you would create a topic for a lamp in your home office:



Note: topics are case-sensitive, which makes these two topics different:



If you would like to turn on a lamp in your home office using MQTT you can imagine the following scenario:

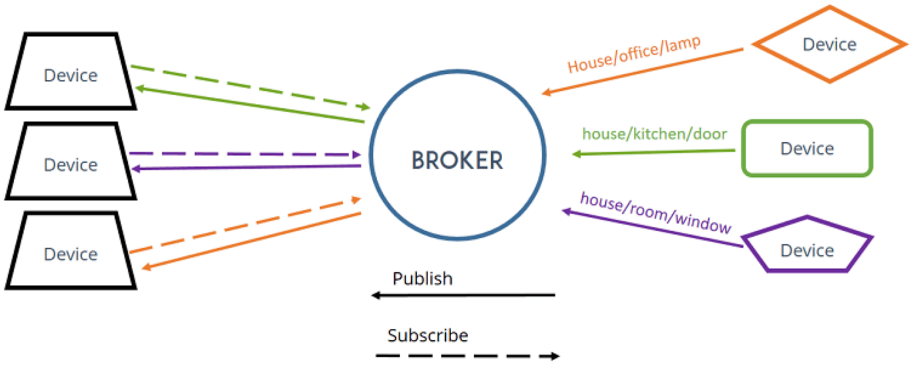


1. You have a device that publishes "on" and "off" messages on the **home/office/lamp** topic.
2. You have a device that controls a lamp (it can be an ESP32, ESP8266, or any other board). The ESP32 that controls your lamp, is subscribed to that topic: **home/office/lamp**.
3. So, when a new message is published on that topic, the ESP32 receives the "on" or "off" message and turns the lamp on or off.

MQTT – Broker

At last, you also need to be aware of the term *broker*.

The broker is primarily responsible for **receiving** all messages, **filtering** the messages, **decide** who is interested in them and then **publishing** the message to all subscribed clients.



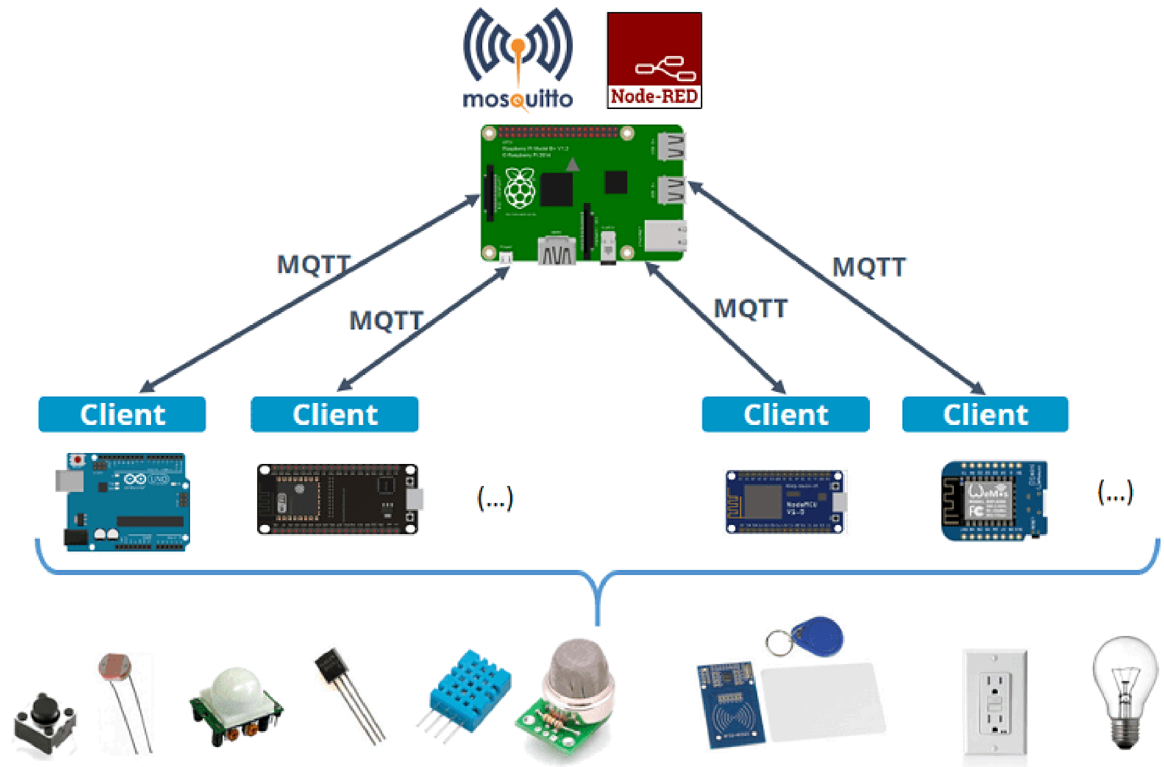
There are several brokers you can use. In our home automation projects we use the Mosquitto broker which can be installed in the Raspberry Pi. Alternatively, you can use a cloud MQTT broker.

To install Mosquitto broker on the Raspberry Pi follow our tutorial:

- How to Install Mosquitto Broker on Raspberry Pi

How to Use MQTT in Home Automation and IoT Projects

As we've seen previously, MQTT is great for home automation and internet of things projects. If you want to start making your own projects using MQTT here's an example of what you can do.



Here's the steps you should follow:

1) Set up your Raspberry Pi. Follow our Getting Started Guide with Raspberry Pi .
2) Enable and Connect your Raspberry Pi with SSH .
3) You need Node-RED installed on your Pi and Node-RED Dashboard .
4) Install the Mosquitto broker on the Raspberry Pi .

5) Add the ESP8266 or the ESP32 to this system. You can follow the next MQTT tutorials:

- **ESP32** and Node-RED with MQTT – Publish and Subscribe
- **ESP8266** and Node-RED with MQTT – Publish and Subscribe

If you want to learn more about these subjects, we have a dedicated course on how to create your own Home Automation System using Raspberry Pi, ESP8266, Arduino, and Node-RED. Just click the following link.

Wrapping up

MQTT is a communication protocol based on a publish and subscribe system. It is simple to use and it is great for Internet of Things and Home Automation projects. We hope you've found this tutorial useful and you now understand what is MQTT and how it works.

SMQTT (Secure Message Queue Telemetry Transport)

SMQTT (Secure Message Queue Telemetry Transport) is an extension of MQTT protocol which uses encryption based on lightweight attribute encryption. The main advantage of this encryption is that it has a broadcast encryption feature. In this features, one message is encrypted and delivered to multiple other nodes.

The process of message transfer and receiving consists of four major stages:

1. **Setup:** In this phase, the publishers and subscribers register themselves to the broker and get a secret master key.
2. **Encryption:** When the data is published to broker, it is encrypted by broker.
3. **Publish:** The broker publishes the encrypted message to the subscribers.
4. **Decryption:** Finally the received message is decrypted by subscribers with the same master key.

SMQTT is proposed only to enhance MQTT security feature

REFERENCES:-

SUMMARY:-

TOPIC:- **CoAP, CoAP message types**

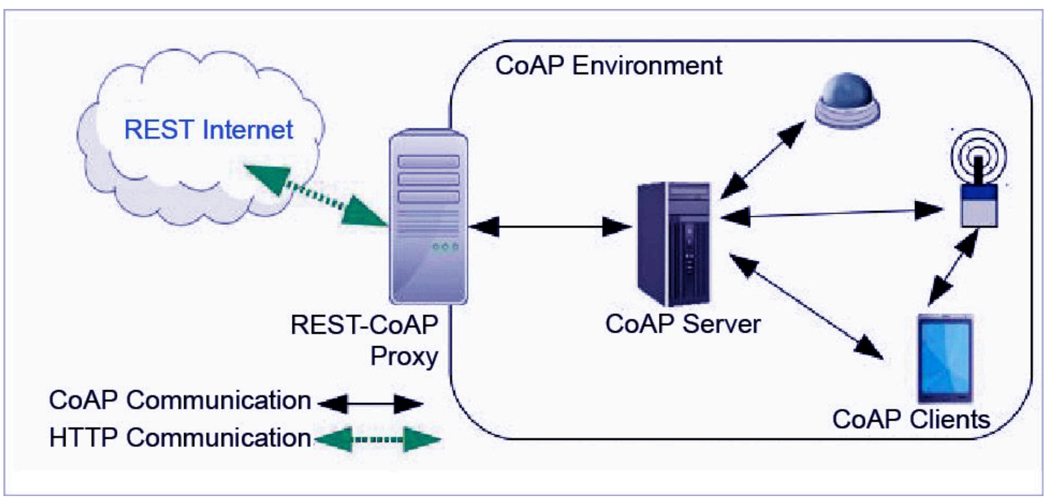
OBJECTIVES:- It is a session layer protocol that provides the RESTful (HTTP) interface between HTTP client and server. This protocol is specially built for IoT systems primarily based on HTTP protocols.

OUTCOME:-

CoAP, CoAP message types

CoAP (Constrained Application Protocol) is a session layer protocol that provides the RESTful (HTTP) interface between HTTP client and server. It is designed by IETF Constrained RESTful Environment (CoRE) working group. It is designed to use devices on the same constrained network between devices and general nodes on the Internet. CoAP enables low-power sensors to use RESTful services while meeting their low power constraints. This protocol is specially built for IoT systems primarily based on HTTP protocols.

This network is used within the limited network or in a constrained environment. The whole architecture of CoAP consists of CoAP client, CoAP server, REST CoAP proxy, and REST internet.



The data is sent from CoAP clients (such as smartphones, RFID sensors, etc.) to the CoAP server and the same message is routed to REST CoAP proxy. The REST CoAP proxy interacts outside the CoAP environment and uploads the data over REST internet.

Constrained Application Protocol (**CoAP**) is a specialized Internet Application Protocol for constrained devices, as defined in RFC 7252. It enables those constrained devices called "nodes" to communicate with the wider Internet using similar protocols. CoAP is designed for use between devices on the same constrained network (e.g., low-power, lossy networks), between devices and general nodes on the Internet, and between devices on different constrained networks both joined by an internet. CoAP is also being used via other mechanisms, such as SMS on mobile communication networks.

Message formats

The smallest CoAP message is 4 bytes in length, if omitting Token, Options and Payload. CoAP makes use of two message types, requests and responses, using a simple, binary, base header format. The base header may be followed by options in an optimized Type-Length-Value format. CoAP is by default bound to UDP and optionally to DTLS , providing a high level of communications security.
Any bytes after the headers in the packet are considered the message body. The length of the message body is implied by the datagram length. When bound to UDP, the entire message MUST fit within a single datagram. When used with 6LoWPAN as defined in RFC 4944, messages SHOULD fit into a single IEEE 802.15.4 frame to minimize fragmentation.

Table 3 Message Format

0 1 2 3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1

Ver	T	OC	Code	MessageID
Token (if any, TKL bytes)…				
Options (if any)…				
Payload (if any)…				

REFERENCES:-

SUMMARY:-

TOPIC:- **CoAP Request-Response model**

OBJECTIVES:- CoAP is a simple protocol with low overhead specifically designed for constrained devices (such as microcontrollers) and constrained networks.

OUTCOME:-

CoAP Protocol

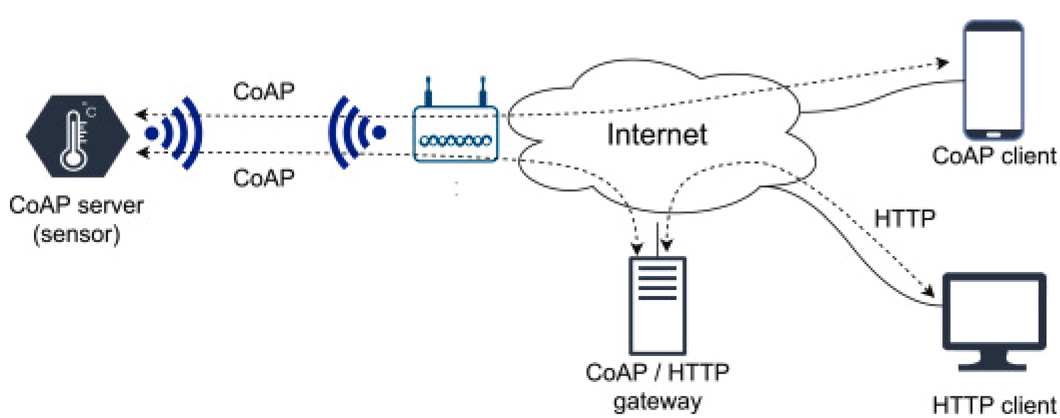
As said before, CoAP is an IoT protocol. CoAP stands for Constrained Application Protocol, and it is defined in RFC 7252 . CoAP is a simple protocol with low overhead specifically designed for constrained devices (such as microcontrollers) and constrained networks. This protocol is used in M2M data exchange and is very similar to HTTP, even if there are important differences that we will cover laters.

The main features of CoAP protocols are:

- Web protocol used in M2M with constrained requirements
- Asynchronous message exchange
- Low overhead and very simple to parse
- URI and content-type support
- Proxy and caching capabilities

As you may notice, some features are very similar to HTTP even if CoAP must not be considered a compressed HTTP protocol because CoAP is specifically designed for IoT and in more details for M2M so it is very optimized for this task.

From the abstraction protocol layer, CoAP can be represented as:



As you can see there are two different layers that make CoAp protocol: Messages and Request/Response. The Messages layer deals with UDP and with asynchronous messages. The Request/Response layer manages request/response interaction based on request/response messages.

CoAP supports four different message types:

- Confirmable
- Non-confirmable
- Acknowledgment
- Reset

Before going deeper into the CoAp protocol, structure is useful to define some terms that we will use later:

- **Endpoint**: An entity that participates in the CoAP protocol. Usually, an Endpoint is identified with a host

- **Sender**: The entity that sends a message

- **Recipient**: The destination of a message

- **Client**: The entity that sends a request and the destination of the response

- **Server**: The entity that receives a request from a client and sends back a response to the client

CoAP Messages Model

This is the lowest layer of CoAP. This layer deals with UDP exchanging messages between endpoints. Each CoAP message has a unique ID; this is useful to detect message duplicates. A CoAP message is built by these parts:

- A binary header
- A compact options
- Payload

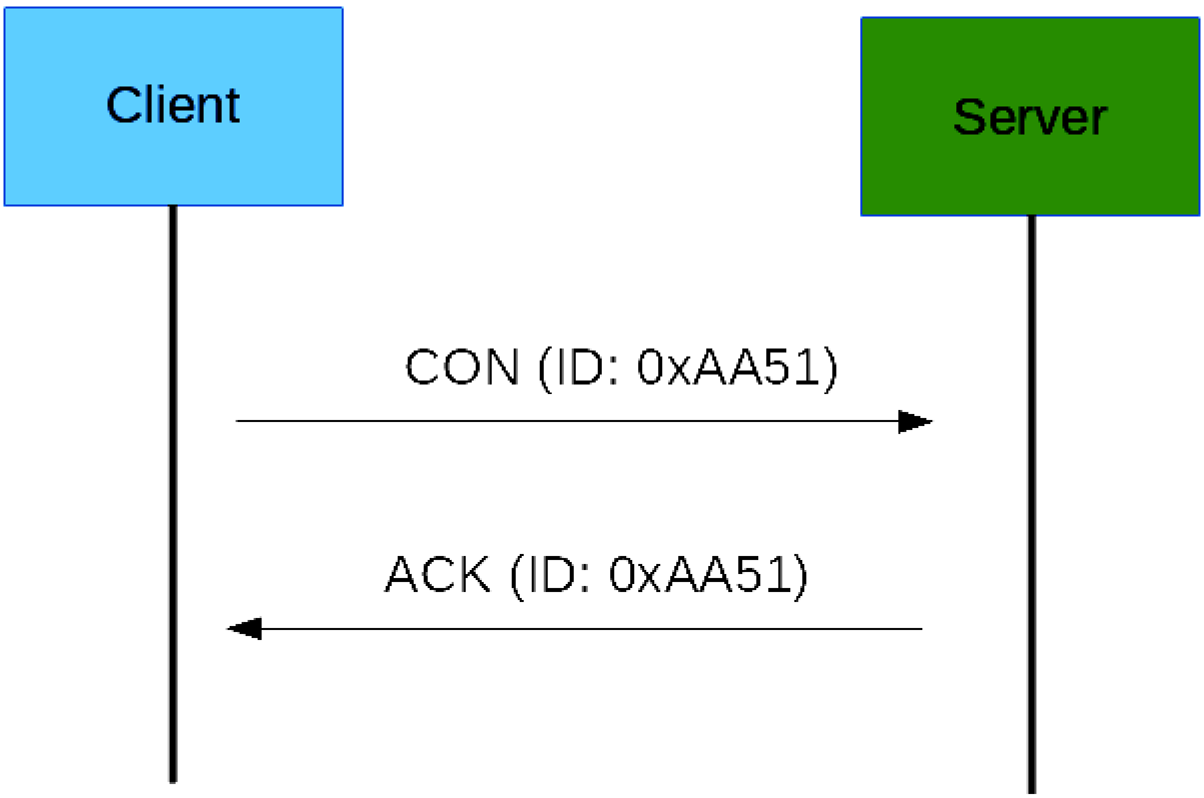
Later, we will describe the message format in more details.

As said before, the CoAP protocol uses two kinds of messages:

- Confirmable message
- Non-confirmable message

A confirmable message is a reliable message. When exchanging messages between two endpoints, these messages can be reliable. In CoAP, a reliable message is obtained using a Confirmable message (CON). Using this kind of message, the client can be sure that the message will arrive at the server. A Confirmable message is sent again and again until the other party sends an acknowledge message (ACK). The ACK message contains the same ID of the confirmable message (CON).

The picture below shows the message exchange process:



If the server has troubles managing the incoming request, it can send back a Rest message (RST) instead of the Acknowledge message (ACK):

The other message category is the Non-confirmable (NON) messages. These are messages that don't require an Acknowledge by the server. They are unreliable messages or in other words messages that do not contain critical information that must be delivered to the server. To this category belongs messages that contain values read from sensors.

Even if these messages are unreliable, they have a unique ID.

CoAp Request/Response Model

The CoAP Request/Response is the second layer in the CoAP abstraction layer. The request is sent using a Confirmable (CON) or Non-Confirmable (NON) message. There are several scenarios depending on if the server can answer immediately to the client request or the answer if not available.

If the server can answer immediately to the client request, then if the request is carried using a Confirmable message (CON), the server sends back to the client an Acknowledge message containing the response or the error code:

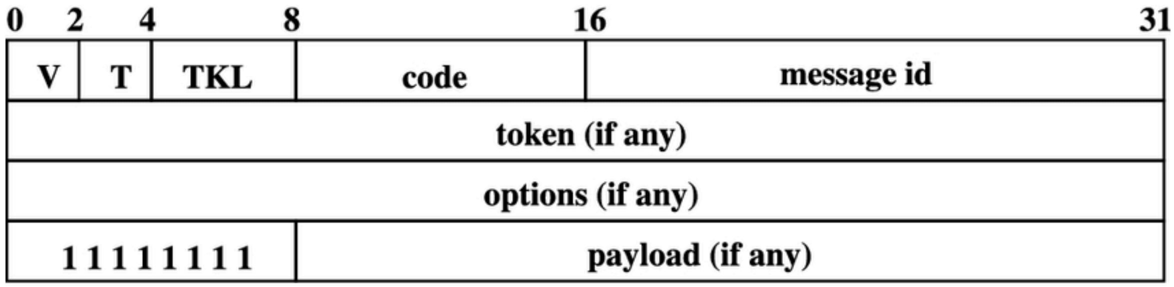
As you can notice in the CoAP message, there is a Token. The Token is different from the Message-ID and it is used to match the request and the response.

If the server can't answer to the request coming from the client immediately, then it sends an Acknowledge message with an empty response. As soon as the response is available, then the server sends a new Confirmable message to the client containing the response. At this point, the client sends back an Acknowledge message:

If the request coming from the client is carried using a NON-confirmable message, then the server answer using a NON-confirmable message.

CoAp Message Format

This paragraph covers the CoAP Message format. By now, we have discussed different kinds of messages exchanged between the client and the server. Now it is time to analyze the message format. The constrained application protocol is the meat for constrained environments, and for this reason, it uses compact messages. To avoid fragmentation, a message occupies the data section of a UDP datagram. A message is made by several parts:



V **version**

T **type**

TKL **token length**

Where:

- **Ver**: It is a 2 bit unsigned integer indicating the version
- **T**: it is a 2 bit unsigned integer indicating the message type: 0 confirmable, 1 non-confirmable
- **TKL**: Token Length is the token 4 bit length
- **Code**: It is the code response (8 bit length)
- **Message ID**: It is the message ID expressed with 16 bit
- And so on.

CoAP Security Aspects

One important aspect when dealing with IoT protocols is the security aspects. As stated before, CoAP uses UDP to transport information. CoAP relies on UDP security aspects to protect the information. As HTTP uses TLS over TCP, CoAP uses *Datagram TLS over UDP.* DTLS supports RSA, AES, and so on. Anyway, we should consider that in some constrained devices some of DTLS cipher suits may not be available. It is important to notice that some cipher suites introduces some complexity and constrained devices may not have resources enough to manage it.

CoAP Vs. MQTT

An important aspect to cover is the main differences between CoAP and MQTT. As you may know, MQTT is another protocol widely used in IoT. There are several differences between these two protocols. The first aspect to notice is the different paradigm used. MQTT uses a publisher-subscriber while CoAP uses a request-response paradigm. MQTT uses a central broker to dispatch messages coming from the publisher to the clients. CoAP is essentially a one-to-one protocol very similar to the HTTP protocol. Moreover, MQTT is an event-oriented protocol while CoAP is more suitable for state transfer.

REFERENCES:-

SUMMARY:-

TOPIC:- **XMPP, AMQP features and components**

XMPP

Extensible Messaging and Presence Protocol (**XMPP**, originally named **Jabber**) is an open communication protocol designed for instant messaging (IM), presence information, and contact list maintenance. Based on XML (Extensible Markup Language), it enables the near-real-time exchange of structured data between any two or more network entities. Designed to be extensible, the protocol offers a multitude of applications beyond traditional IM in the broader realm of message-oriented middleware, including signalling for VoIP, video, file transfer, gaming and other uses.
Unlike most commercial instant messaging protocols, XMPP is defined in an open standard in the application layer. The architecture of the XMPP network is similar to email; anyone can run their own XMPP server and there is no central master server. This federated open system approach allows users to interoperate with others on any server using a 'JID' user account, similar to an email address. XMPP implementations can be developed using any software license and many server, client, and library implementations are distributed as free and open-source software. Numerous freeware and commercial software implementations also exist.

Protocol characteristics

A simple XMPP network with the servers *jabber.org* and *draugr.de*. Green clients are online, yellow clients are writing each other and small green *subclients* are the resources of one user. The brown network is not connected to the internet. The server *draugr.de* is connected to other IM services (ICQ, AIM and other) via *XMPP transports*.

The XMPP network architecture is reminiscent of the Simple Mail Transfer Protocol (SMTP), a client–server model; clients do not talk directly to one another as it is decentralized - anyone can run a server. By design, there is no central authoritative server as there is with messaging services such as AIM, WLM, WhatsApp or Telegram. Some confusion often arises on this point as there is a public XMPP server being run at jabber.org, to which many users subscribe. However, anyone may run their own XMPP server on their own domain.

Addressing

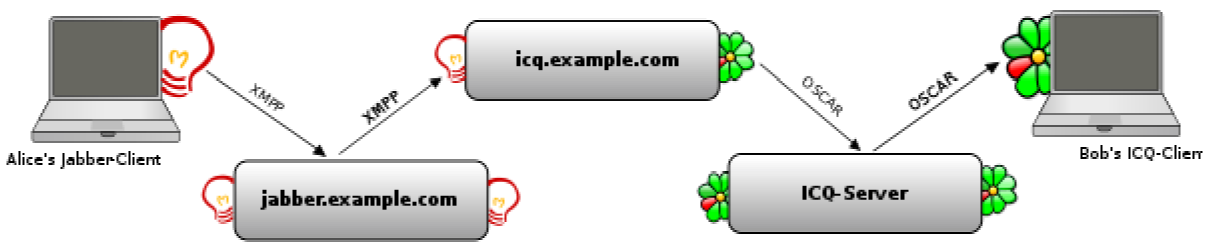
```
alice@example.com/home
 node      domain    resource
```

A standard JID

Every user on the network has a unique XMPP address, called *JID*[5] (for historical reasons, XMPP addresses are often called *Jabber IDs*). The JID is structured like an email address with a username and a domain name (or IP address) for the server where that user resides, separated by an at sign (@) - for example, “alice@example.com“: here alice is the username and example.com the server with which the user is registered.

Since a user may wish to log in from multiple locations, they may specify a **resource**. A resource identifies a particular client belonging to the user (for example home, work, or mobile). This may be included in the JID by appending a slash followed by the name of the resource. For example, the full JID of a user's mobile account could be username@example.com/mobile.

Connecting to other protocols



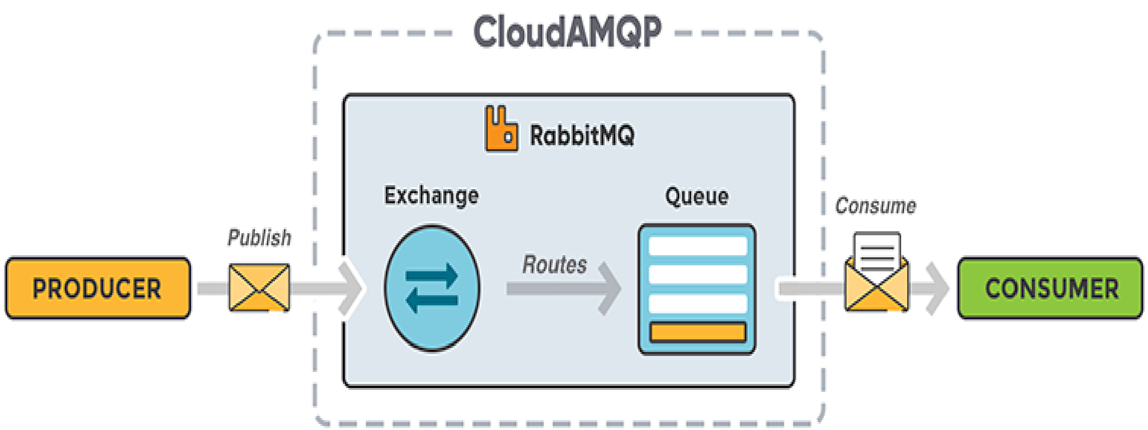
Alice sends a message through the XMPP net to the ICQ transport. The message is next routed to Bob via the ICQ network.

The Advanced Message Queuing Protocol (**AMQP**)

It is an open standard for passing business messages between applications or organizations. It connects systems, feeds business processes with the information they need and reliably transmits onward the instructions that achieve their goals.

AMQP stands for Advanced Message Queuing Protocol and it is an open standard application layer protocol. RabbitMQ implements version 0-9-1 of the specification today, with legacy support for version 0-8 and 0-9. AMQP was designed to efficiently support a wide variety of messaging applications and communication patterns.

As with other message queuing protocols, the defining features of AMQP are message orientation and queuing. **Routing** is another feature, the process by which an exchange decides which queues to place your message on. Messages in RabbitMQ are routed from the exchange to the queue depening on exchange types and keys. **Reliability and Security** are other important features of AMQP. RabbitMQ can be configured to ensure that messages are always delivered, read more in the Reliability Guide.
Getting started guides on CloudAMQP and our language documentation shows sample code using RabbitMQ with the AMQP protocol. For a more complete guide to AMQP see RabbitMQ's concept guide.
For more information about AMQP, check out the AMQP Working Group's overview page.



AMQP Components

- Exchange: A part of the broker (i.e. server) which receives messages and routes them to queues.
- Queue (message queue): A named entity which messages are associated with and from where consumers receive them.
- Bindings: Rules for distributing messages from exchanges to queues.

Message format

AMQP defines as the *bare message*, that part of the message that is created by the sending application. This is considered immutable as the message is transferred between one or more processes.

Ensuring the message as sent by the application is immutable allows for end-to-end message signing and/or encryption and ensures that any integrity checks (e.g. hashes or digests) remain valid. The message can be annotated by intermediaries during transit, but any such annotations are kept distinct from the immutable *bare message*. Annotations may be added before or after the bare message.

The *header* is a standard set of delivery-related annotations that can be requested or indicated for a message and includes time to live, durability, priority.

The bare message itself is structured as an optional list of standard properties (message id, user id, creation time, reply to, subject, correlation id, group id etc.), an optional list of application-specific properties (i.e., extended properties) and a body, which AMQP refers to as application data.

Properties are specified in the AMQP type system, as are annotations. The application data can be of any form, and in any encoding the application chooses. One option is to use the AMQP type system to send structured, self-describing data.

REFERENCES:-

SUMMARY:-

TOPIC:- **AMQP frame types**

AMQP frame types

MESSAGES CONTENT FRAMING

AMQP builtin type system, I’d like to show you a different sample frame strictly related to the sending messages. For starting, I used the SASL mechanisms frame as example to understand the encoding/decoding system just because it’s one of the first exchanged frames in the protocol but every day we use AMQP to send messages so understand their encoding is much more important.

An AMQP message on the wire

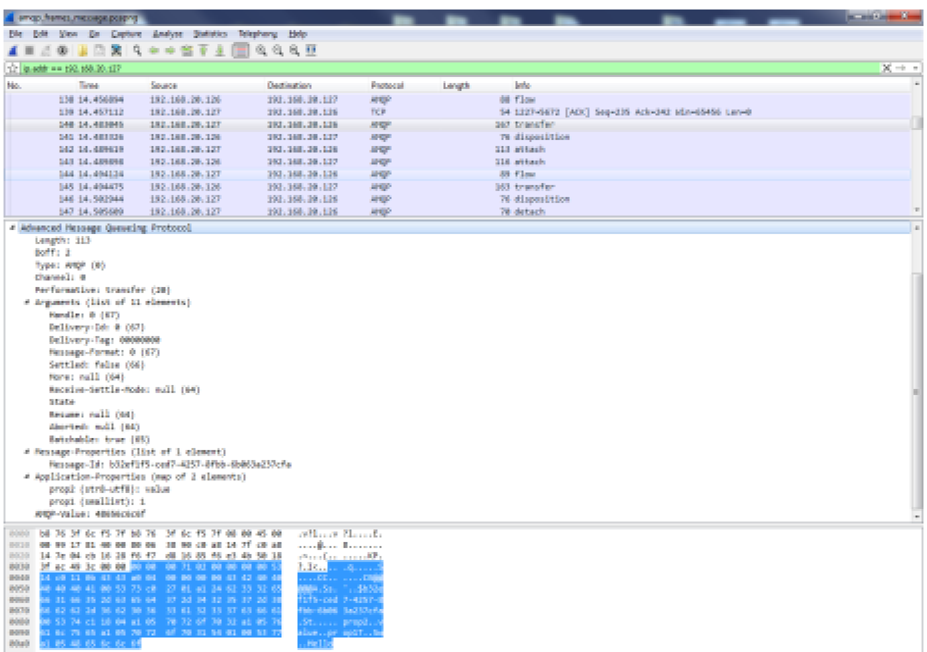
Consider the following source code that uses AMQP .Net Lite library to send a simple message :

```
Connection connection = new Connection(address);
Session session = new Session(connection);
SenderLink sender = new SenderLink(session, "sender-" + testName, "q1");

Message message = new Message("Hello");
message.Properties = new Properties();
message.Properties.MessageId = Guid.NewGuid().ToString();
message.ApplicationProperties = new ApplicationProperties();
message.ApplicationProperties["prop1"] = 1;
message.ApplicationProperties["prop2"] = "value";

sender.Send(message, 60000);
```

In the above code, we are interested in the encoding and framing used in the AMQP protocol to send the *Message* object on the wire. As reported on the official specification (page 38), each frame has a format as showed in the following picture (we already saw this format for the SASL frames with some “little” changes).

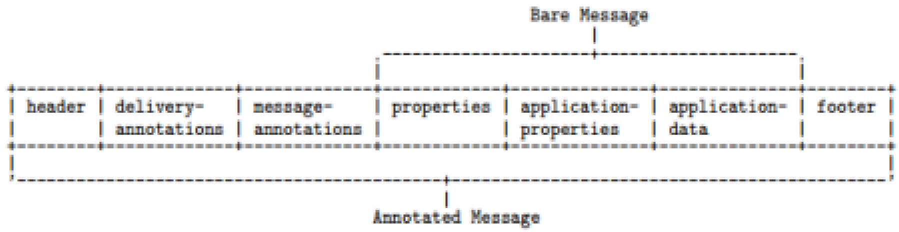
```
            type: 0x00 - AMQP frame

              +0       +1       +2       +3
         +-----------------------------------+ -.
       0 |                SIZE               |  |
         +-----------------------------------+  |---> Frame Header
       4 |  DOFF  |  TYPE  |     CHANNEL     |  |      (8 bytes)
         +-----------------------------------+ -'
         +-----------------------------------+ -.
       8 |                ...                |  |
         .                                   .  |---> Extended Header
         .             <IGNORED>             .  |  (DOFF * 4 - 8) bytes
         |                ...                |  |
         +-----------------------------------+ -'
         +-----------------------------------+ -.
  4*DOFF |           PERFORMATIVE:           |  |
         .      Open / Begin / Attach        .  |
         .   Flow / Transfer / Disposition   .  |
         .      Detach / End / Close         .  |
         |-----------------------------------|  |
         .                                   .  |---> Frame Body
         .                                   .  |  (SIZE - DOFF * 4) bytes
         .             PAYLOAD               .  |
         .                                   .  |
         .                         __________|  |
         |             ...        |             |
         +------------------------+            -'
```

As we can see, the real body starts with the so called **performative**; you can think about it as an “operation” executed against the AMQP broker (opening the connection or the session, attaching the link, transfering a message and so on).



In this case, we have the **transfer** performative for sending messages and the encoded frame is much more complex than the first one (SASL mechanisms).

The objective of this article is to understand how the builtin type system is used to encode the three main parts of an AMQP message.



From the above picture we'll consider :

- **Message properties** : system properties well defined by the AMQP specification and used by the broker for manipulation, routing and so on;
- **Application properties** : user defined properties to carry data without using the following payload;
- **Application data** : user defined payload of the message;

The content of all these parts is encoded using the builtin type system. Starting from the "Message-Properties" section we have the descriptor (0x00 0x53 0x73) where the 0x73 code defines the **"AMQP properties list"** (page 84 of official specification) that is a list (format code 0xC0) of 39 bytes (0x27) with only one element (0x01). This element is an UTF8 encoded string (0xA1) with size of 36 characters expressed with only one byte (0x24). This string is the **"Message-Id"** we set in our source code (it's the autogenerated GUID value). You can see that there is no information to describe that this field is the "Message-Id" field but we know that because it's the first field of the composite type "AMQP properties list". If the "Message-Id" field weren't explicitly set by code, it would be null but encoded inside the properties list itself. This means that all missing fields between two defined fields are encoded as null but inserted inside the list (to guarantee the fields sequence); if all subsequent fields (until the end of the list) aren't set, they aren't encoded inside the frame (to avoid a waste of space).

```
◢ Message-Properties (list of 1 element)
    Message-Id: b32ef1f5-ced7-4257-8fbb-6b063a237cfa
▷ Application-Properties (map of 2 elements)
  AMQP-Value: 48656c6c6f
```

```
0000  b8 76 3f 6c f5 7f b8 76  3f 6c f5 7f 08 00 45 00   .v?l...v ?l....E.
0010  00 99 17 81 40 00 80 06  38 90 c0 a8 14 7f c0 a8   ....@... 8.......
0020  14 7e 04 cb 16 28 f6 f7  d8 16 85 f6 e3 4b 50 18   .~...(.. .....KP.
0030  3f ec 49 3c 00 00 00 00  00 71 02 00 00 00 00 53   ?.I<.... .q.....S
0040  14 c0 11 0b 43 43 a0 04  00 00 00 00 43 42 40 40   ....CC.. ....CB@@
0050  40 40 40 41 00 53 73 c0  27 01 a1 24 62 33 32 65   @@@A.Ss. '..$b32e
0060  66 31 66 35 2d 63 65 64  37 2d 34 32 35 37 2d 38   f1f5-ced 7-4257-8
0070  66 62 62 2d 36 62 30 36  33 61 32 33 37 63 66 61   fbb-6b06 3a237cfa
0080  00 53 74 c1 18 04 a1 05  70 72 6f 70 32 a1 05 76   .St..... prop2..v
0090  61 6c 75 65 a1 05 70 72  6f 70 31 54 01 00 53 77   alue..pr op1T..Sw
00a0  a1 05 48 65 6c 6c 6f                               ..Hello
```

The “Application-Properties” section starts with its related descriptor (0x00 0x53 0x74) where the 0x74 code defines the **“Application properties map”** (page 86 of official specification) that is a map (format code 0xC1) of 24 bytes (0x18) with four elements (0x04). A **map** is a compound type in the AMQP builtin type system we didn’t see yet; it’s a collection with key-value pairs each encoded as defined in the specification.

```
 item 0   item 1      item n-1    item n
+-------+-------+----+---------+---------+
| key 1 | val 1 | .. | key n/2 | val n/2 |
+-------+-------+----+---------+---------+
```

In our example, we have two pairs (“prop1” = 1 and “prop2” = “value”) so four elements in the map. The name of the first key “prop1” is encoded as a UTF8 string (code 0xA1) with 0x05 a size (five characters for “prop1”) and the related value that is the string “value” is encoded in the same way. Then there is the name of the second key “prop2” and the related value 0x01 that is an integral type expressed with only one byte (format code 0x54, page 27 on official specification).

```
◢ Application-Properties (map of 2 elements)
    prop2 (str8-utf8): value
    prop1 (smallint): 1
  AMQP-Value: 48656c6c6f
```

```
0000  b8 76 3f 6c f5 7f b8 76  3f 6c f5 7f 08 00 45 00   .v?l...v ?l....E.
0010  00 99 17 81 40 00 80 06  38 90 c0 a8 14 7f c0 a8   ....@... 8.......
0020  14 7e 04 cb 16 28 f6 f7  d8 16 85 f6 e3 4b 50 18   .~...(.. .....KP.
0030  3f ec 49 3c 00 00 00 00  00 71 02 00 00 00 00 53   ?.I<.... .q.....S
0040  14 c0 11 0b 43 43 a0 04  00 00 00 00 43 42 40 40   ....CC.. ....CB@@
0050  40 40 40 41 00 53 73 c0  27 01 a1 24 62 33 32 65   @@@A.Ss. '..$b32e
0060  66 31 66 35 2d 63 65 64  37 2d 34 32 35 37 2d 38   f1f5-ced 7-4257-8
0070  66 62 62 2d 36 62 30 36  33 61 32 33 37 63 66 61   fbb-6b06 3a237cfa
0080  00 53 74 c1 18 04 a1 05  70 72 6f 70 32 a1 05 76   .St..... prop2..v
0090  61 6c 75 65 a1 05 70 72  6f 70 31 54 01 00 53 77   alue..pr op1T..Sw
00a0  a1 05 48 65 6c 6c 6f                               ..Hello
```

The last part of the message is the data section (the “Hello” string) that has the descriptor (0x00 0x53 0x77) where the code 0x77 defines an **“AMQP value”** (page 86 of official specification) that in this case is an UTF8 string (0xA1) with 0x05 characters.

```
AMQP-Value: 48656c6c6f

0000  b8 76 3f 6c f5 7f b8 76  3f 6c f5 7f 08 00 45 00   .v?l...v ?l....E.
0010  00 99 17 81 40 00 80 06  38 90 c0 a8 14 7f c0 a8   ....@... 8.......
0020  14 7e 04 cb 16 28 f6 f7  d8 16 85 f6 e3 4b 50 18   .~...(.. .....KP.
0030  3f ec 49 3c 00 00 00 00  00 71 02 00 00 00 00 53   ?.I<.... .q.....S
0040  14 c0 11 0b 43 43 a0 04  00 00 00 00 43 42 40 40   ....CC.. ....CB@@
0050  40 40 40 41 00 53 73 c0  27 01 a1 24 62 33 32 65   @@@A.Ss. '..$b32e
0060  66 31 66 35 2d 63 65 64  37 2d 34 32 35 37 2d 38   f1f5-ced 7-4257-8
0070  66 62 62 2d 36 62 30 36  33 61 32 33 37 63 66 61   fbb-6b06 3a237cfa
0080  00 53 74 c1 18 04 a1 05  70 72 6f 70 32 a1 05 76   .St..... prop2..v
0090  61 6c 75 65 a1 05 70 72  6f 70 31 54 01 00 53 77   alue..pr op1T..Sw
00a0  a1 05 48 65 6c 6c 6f                               ..Hello
```

The AMQP message format seems to be more complex than any other protocol but deeping into it …. it’s seems to be simpler.

Of course, there is another part in the message we didn’t consider named the “Message Annotations” that are used a lot for customization inside brokers like for Event Hubs inside Microsoft Azure Service Bus.

However, we can’t see them on the wire because we know that Service Bus traffic is encrypted (using SSL/TLS protocol) and the local AMQP broker for testing doesn’t support event hubs for clear reasons

REFERENCES:-

SUMMARY:-